

gaugefit — Integration Guide

How to put gaugefit inside your own application. Written for the developer who has to make it work on a line, not for someone reading about metrology. Current as of version 0.1.0.

A Korean edition of this document is at [docs/integration.ko.md](https://docs.integration.ko.md); the content is the same.

1. What gaugefit does for you

You give it an image and a rectangle laid across an edge. It tells you where that edge is, to a few thousandths of a pixel, and it tells you when not to believe the answer.

On top of that single measurement there are the things a real inspection needs:

- **Pairing policies** — which two edges make the width, stated explicitly rather than guessed. Most false measurements in the field come from this choice being hidden.
- **Line and Circle finders** — many calipers, individual failures absorbed as outliers and reported, never quietly dropped.
- **Metrology** — distances, angles, parallelism, roundness, straightness between fitted shapes.
- **Fixturing** — a transform so the whole setup follows the part instead of the frame.
- **Calibration** — pixels to millimetres, applied to the measured points, never by resampling the image.
- **An error budget** — the uncertainty that goes with the number, backed by measured bias curves rather than a rule of thumb.

The only runtime dependency is numpy — engine, finders, recipe runtime, all of it. Two setup-time features ask for more: calibrating from a dot-grid target and the interaction F-test in Gage R&R want SciPy (`pip install "gaugefit[calib]"`), and image formats beyond PNG/PGM want Pillow (`gaugefit[image]`). Neither is needed to import the package or to measure anything.

2. Install

```
pip install gaugefit
```

Python 3.10 - 3.14 on Windows x86-64, Linux x86-64 / aarch64, macOS arm64.

```
import gaugefit as gauge
gauge.engine_info()
# {'engine': 'native', 'native_available': True, 'simd': 'AVX2+FMA', 'threads': 32, ...}
```

Images are 2-D float arrays. Anything numpy-shaped works; `gauge.imread()` handles PNG and PGM with no extra dependency and other formats through Pillow if it is installed. Values may be 0-1 or 0-255 — the caliper only cares about differences, but `contrast` thresholds are in whatever units you feed it, so pick one and stay with it.

3. The five-minute version

```
import gaugefit as gauge
```

```
img = gauge.imread("part.png")
cal = gauge.Caliper(center=(320, 240), angle_deg=0, length=90, width=20)
res = cal.measure(img)
for e in res.edges:
    print(e.t, e.point, e.polarity, e.contrast)
pair = cal.measure_pair(img, policy="first_last", pair_polarity="opposite")
print(pair.width)
```

`t` is the position along the measurement axis, measured from the ROI centre. `point` is the same thing in image coordinates. That is the whole core.

4. Coordinate convention (read this once)

- **Pixel centres are at $(i + 0.5, j + 0.5)$.** The corner of pixel (0,0) is the origin. If your other library puts centres at integers you are half a pixel out, systematically.
- **Angles are degrees, positive counter-clockwise on screen** (y increases downward, so this is clockwise in a maths sense — it matches what you see).
- **`angle_deg` is the direction the caliper measures in.** The ROI extends `length/2` each way along it, and `width/2` each way perpendicular.
- **`polarity` is read along the measurement direction.** `dark_to_light` means the profile rises as `t` increases. Turn the caliper 180° and the polarity of the same physical edge flips. This trips everyone up once.
- **For `LineFinder`, each caliper measures perpendicular to the segment `p0` → `p1`,** along the segment direction rotated +90°. Polarity is read in that direction.
- **For `AnnularCaliper` and `CircleFinder`, `t` is radius** and polarity is read outward. A bright disk on a dark ground is `light_to_dark`.

Same convention as geofit, so poses pass between them unchanged.

5. One caliper, properly

```
cal = gauge.Caliper(
    center=(320, 240),
    angle_deg=0,          # measurement direction
    length=90,            # how far to look, along the measurement axis
    width=20,             # projection width, perpendicular to it
    sigma=1.5,            # smoothing before differentiating
    contrast=0.05,        # minimum step height, in image units
    polarity="any",       # "dark_to_light" | "light_to_dark"
    interp="bspline3",    # resampling
    subpixel="gauss3",    # peak estimator
    project="mean",       # "mean" | "median" | "trimmed"
)
```

Choosing the numbers:

parameter	how to pick it
<code>length</code>	just long enough to contain the edge under every part position. Longer means more chances to catch a neighbouring edge.

parameter	how to pick it
<code>width</code>	as wide as the edge stays straight. Wider averages down noise as $1/\sqrt{W}$, but $W^2/12$ of curvature bias appears on a curved edge, and $W \cdot \tan\Delta$ smears if the ROI is misaligned. Keep $W \cdot \tan\Delta < 3\sigma$.
<code>sigma</code>	1.5 is a good default. Raise it for noisy or soft edges; it costs almost nothing in time. Adjacent edges interfere out to 5σ , so σ also sets how close two edges may be.
<code>contrast</code>	measure a good image and set it to about half the real step height.
<code>polarity</code>	set it whenever you can. It is the cheapest way to stop a caliper latching onto the wrong edge.

`gauge.suggest(img)` reads an image and proposes `sigma`, `contrast`, `length`, `width`, `polarity`, and a `why` string explaining each. It is a starting point, not an answer.

6. Reading the result

```
res = cal.measure(img)
res.edges      # list of MeasuredEdge, in profile order
res.profile    # the 1-D projected profile
res.derivative # the smoothed first derivative
res.t          # profile coordinate for each sample
res.flags      # set of warnings
res.diag       # dict of diagnostics
```

`MeasuredEdge` carries `t`, `point`, `index`, `amp`, `contrast`, `polarity` and `sigma_total` (the total edge width, in profile samples).

Flags are the part you must handle.

flag	what it means	what to do
<code>LOW_CONTRAST</code>	nothing crossed the contrast threshold	check lighting, or the ROI is in the wrong place
<code>NEAR_BOUNDARY</code>	an edge sits inside the filter margin at the end of the profile	make the ROI longer, or move it
<code>EDGES_TOO_CLOSE</code>	two edges nearer than 5σ — their peaks pull on each other	lower σ , or accept the bias
<code>SATURATED</code>	the profile hit the declared saturation level	reduce exposure; a clipped edge is biased, not just noisy
<code>TILTED</code>	the edge is not perpendicular to the measurement axis by more than the projection can absorb	rotate the ROI, or narrow it

`diag` carries `n_peaks`, `best_rejected` (the strongest thing that failed the contrast test — this is what tells you why nothing was found), `sigma_total`, `psf_sigma_est` (the lens blur backed out of the edge width), and, when `tilt_bands > 0`, `tilt_deg`, `smear` and `effective_sigma`.

A caliper that finds nothing is not an exception. It returns an empty `edges` list with flags and `best_rejected` set. Pairing is where refusal becomes an error:

```

from gaugefit import PairingError
try:
    pair = res.pair("first_last", pair_polarity="opposite")
except PairingError as e:
    log.warning("no pair: %s (best rejected %.3f)", e, res.diag["best_rejected"])

```

7. Choosing which edge, on purpose

This is where field false-measurements come from, so the choice is a parameter, not a heuristic.

Single edge — `gauge.select_edge(edges, policy)`:

policy	picks
<code>first</code>	smallest <code>t</code>
<code>last</code>	largest <code>t</code>
<code>strongest</code>	largest <code>abs(amp)</code>
<code>closest_to_center</code>	smallest <code>abs(t)</code>

Pairs — `res.pair(policy, ...)`:

policy	picks
<code>first_last</code>	outermost pair
<code>strongest_pair</code>	the two largest amplitudes
<code>closest_to_nominal</code>	the pair whose width is nearest <code>nominal_width</code>
<code>widest / narrowest</code>	extremes of width

With `pair_polarity="opposite"` (a solid object: rising edge then falling edge), `"same"`, or `None` for either. `tolerance` rejects a `closest_to_nominal` pair that is further off than you are willing to accept — better an error than a confident wrong width.

8. Lines and circles

```

line = gauge.LineFinder(
    p0=(100, 80), p1=(100, 400), # the segment the calipers sit along
    n=15,                        # how many
    length=40, width=10,         # each caliper's ROI
    edge_policy="strongest",
    robust="tukey", ransac=True, # outlier handling
).run(img)
line.ok          # False when it could not fit
line.shape       # Line
line.points      # the edge points that were used
line.results     # every caliper's raw CaliperResult, in order, failures included
line.used, line.failed
line.rms, line.span # span is straightness: peak-to-peak inlier residual
line.inliers
line.flags       # union of caliper flags + MANY_CALIPER_FAILURES / MANY_OUTLIERS

```

CircleFinder is the same shape of thing, with annular calipers:

```
circ = gauge.CircleFinder(center=(300, 300), r_nominal=61, n=24,
                          r_span=30, arc=8).run(img)
circ.shape.center, circ.shape.radius
circ.span                      # roundness
circ.diag["curvature_correction"]
```

Why annular and not linear ROIs on a circle. A straight ROI measuring a curved edge reads short by $-(\sigma_{\text{psf}}^2 + W^2/12) / 2R$ — the chord effect plus the 2-D blur. At $R = 60$, $W = 20$ that is 0.28 px on the radius, which is a real diameter error. An annular ROI projects along the arc, so the $W^2/12$ term disappears. The remaining optical term is corrected automatically: σ_{psf} is backed out of the measured edge width. Pass **psf_sigma**= if you know it, or **curvature_correction=False** to turn it off.

results is not decoration. When a line reads wrong, the first thing to look at is which caliper contributed what:

```
for k, r in enumerate(line.results):
    mark = "OUT" if k in line.failed else ("in" if k in line.used else " ")
    print(k, mark, r.flags, r.diag.get("best_rejected"))
```

9. Fixturing — following the part

Teach the setup once with the part in a reference position. At runtime, tell gaugefit where the part is now, and every ROI moves with it.

```
recipe = gauge.load_recipe("part.gaugefit.json")
# geofit found the part:
out = recipe.run(img, pose=(match.x, match.y, match.angle))
# or directly:
fx = gauge.Fixture.from_geofit(match, ref=(380.0, 260.0, 0.0))
cal_now = fx.apply(cal)
```

pose is (**x**, **y**, **angle_deg**) or anything with **.x** / **.y** / **.angle** — a geofit **Match** drops straight in. The reference pose is stored in the recipe, so the same file works from either side.

Without fixturing, a part that shifts by two pixels shifts every measurement by two pixels. With it, the shift cancels: the bundled sample rolls by (5, 7) px and the measured values move by less than 0.05 px.

10. Millimetres

```
cal = gauge.Calibration.from_scale(px_per_unit=41.7, unit="mm")
# or from a dot-grid target, which also solves lens distortion:
from gaugefit.calib import calibrate_from_dot_grid
cal = calibrate_from_dot_grid(target_img, rows=9, cols=11, pitch=2.0)
world = cal.to_world(points_px)
cal.distance(p_px, q_px)          # a world-unit length between two pixel points
cal.scale_at(p_px)                # local units-per-pixel, if distortion varies it
```

There is no `undistort_image()`, and there will not be. Undistorting an image means resampling it, and resampling destroys exactly the subpixel information the caliper exists to read.

`gaugefit.calib.distortion.undistort_image` is present only to raise `NotImplementedError` with that sentence. Correct the points, after measuring them.

Attach a calibration to a recipe and every measurement comes back in world units, with `out.units` saying which.

11. Handing a setup over as a recipe

```
gaugefit serve
```

Place the tools on a real image, watch the profile and derivative under the ROI as you drag it, add measurements, export. You get one JSON file holding the tools, the measurements, the fixture, the calibration and a base64 reference image. Nothing else is needed to reproduce the setup.

```
# 1) from the exported file
recipe = gauge.load_recipe("part.gaugefit.json")
# 2) live from a dashboard running on this machine – no URL needed
recipe = gauge.fetch_recipe()
# 3) from any JSON text you already have
recipe = gauge.load_recipe(text)
out = recipe.run(img, pose=pose)
out.values      # {"plate_gap": 184.0, "big_dia": 80.0, ...}
out.units       # {"plate_gap": "mm", ...}
out.shapes      # the fitted Line / Circle objects, by tool name
out.tools       # per-tool raw results
out.flags
out.failed      # [(name, "TypeError: ..."), ...] – one tool dying does not stop the rest
```

Tool types: `caliper`, `annular`, `line`, `circle`. Measurement types: `point_point`, `point_line`, `point_circle`, `line_line`, `line_angle`, `parallelism`, `perpendicularity`, `line_circle`, `circle_center_distance`, `circle_diameter`, `circle_radius`, `circle_roundness`, `line_straightness`, `tool_width`, `edge_position`.

`recipe.run()` is the whole runtime API. The dashboard is for setting up; it is not required at runtime and does not have to be installed anywhere near the line.

12. Uncertainty, and winning the argument about it

A repeatability number and an accuracy number are different claims. Customers ask for one and mean the other. The budget keeps them apart:

```
from gaugefit.quality import caliper_budget
b = caliper_budget(sigma=1.5, psf=0.8, angle=0, width=20, snr_db=35)
b.total                                # combined 1σ, px
for t in b.terms:
    print(t.name, t.sigma, t.kind, t.source)
# ■■■■ ■■■■ 0.00107 systematic study B/C (bspline3, psf 0.8, angle 0°)
# ■■■■ ■■■ 0.00003 systematic study A (gauss3, σ 1.5)
# ■■■ ■■■ 0.00843 random study D (gauss3, SNR 35 dB)
```

Every term names the study it came from. Those studies are in `docs/bias.md` and the raw sweep ships as `gaugefit/data/bias_table.csv`, so a disputed number can be traced to a measurement rather than a claim.

For a real gauge study on real parts:

```
from gaugefit.quality import gage_rr
g = gage_rr(data, tolerance=0.05)      # data[part][operator] = [trial, trial, ...]
g.grr, g.ndc, g.ev, g.av, g.pv
```

13. Speed and threads

A caliper (profile 80 × width 48) takes about 58 s; a 24-caliper circle finder with robust fitting takes about 0.16 ms. Where the time goes, in order: resampling, then everything else. Raising σ from 1.5 to 6 costs 6 s — do not economise there.

- Calls release the GIL. Running finders on several images from several Python threads actually parallelises.
- `gauge.set_num_threads(n)` sets the caliper pool. Thread count changes speed only; results are identical at 1, 2, 4, 8 and 32 threads, and there is a test that says so.
- Reuse the prepared image when several tools look at one frame:

```
pre = gauge.prepare(img, "bspline3")
a = cal_a.measure(img, pre)
b = cal_b.measure(img, pre)
```

`recipe.run()` does this for you.

- `interp="bilinear"` is faster and costs about 0.03 px of phase-dependent bias. `bicubic` buys nothing over `bilinear` — measured, not assumed.

14. When something looks wrong

"It found the wrong edge." Set `polarity`. Then narrow `length`. Then use an explicit `edge_policy` / pairing policy instead of the default.

"The width is a few hundredths out, consistently." Look for curvature (use annular), ROI misalignment (`diag["tilt_deg"]`), or an axis-aligned edge — 0° is the worst case for resampling bias, not the best. Tilting the ROI 5° reduces it by an order of magnitude.

"It is noisy." Widen `width` before raising `sigma`; noise falls as $1/\sqrt{w}$ with no bias cost while the edge stays straight.

"Two edges too close." They interfere out to 5σ , not 3σ . Lower σ , or measure them with separate calipers.

"Nothing found." `res.diag["best_rejected"]` is the strongest candidate that failed the contrast test. If it is close to your `contrast`, the threshold is the problem. If it is near zero, the ROI is not on the edge.

"Native library not found." `gauge.engine_info()["load_error"]` says why. You are probably running from a source checkout rather than an installed wheel. The pure-Python engine still works —

`gauge.use_native(False)` — it is just slower.

"Do the two engines agree?" They must. `cal.measure(img)` and `cal.measure_py(img)` should give identical numbers; if they do not, that is a bug worth reporting, with the image.

15. API quick reference

```
import gaugefit as gauge
# engine
gauge.engine_info(); gauge.use_native(False); gauge.set_num_threads(8); gauge.native_available()
# tools
gauge.Caliper(center, angle_deg, length, width, **params).measure(img) -> CaliperResult
gauge.AnnularCaliper(center, r_min, r_max, angle_deg, arc, **params).measure(img)
gauge.LineFinder(p0, p1, n, length, width, **params).run(img) -> FinderResult
gauge.CircleFinder(center, r_nominal, n, r_span, arc, **params).run(img)
gauge.prepare(img, "bspline3") # share across tools on one frame
# choosing edges
gauge.select_edge(edges, "strongest"); res.pair("first_last", pair_polarity="opposite")
gauge.SINGLE_POLICIES, gauge.PAIR_POLICIES, gauge.PairingError
# fitting shapes yourself
gauge.fit_line_robust(pts); gauge.fit_circle_robust(pts)
gauge.fit_line(pts); gauge.fit_circle(pts) # plain, no outlier handling
# metrology
from gaugefit import metrology as mt
mt.point_point, mt.point_line, mt.line_line, mt.circle_circle,
mt.parallelism, mt.perpendicularity, mt.intersect, mt.angle_between
# fixturing and calibration
gauge.Fixture.from_geofit(match, ref); gauge.Fixture.from_poses(ref, cur)
gauge.Calibration.from_scale(px_per_unit, unit="mm")
from gaugefit.calib import calibrate_from_dot_grid
# recipes – hand a whole setup to another program
gauge.load_recipe(path_or_text); gauge.fetch_recipe(); recipe.run(img, pose=None)
gauge.dashboard_url()
# quality
from gaugefit.quality import caliper_budget, gage_rr, repeatability, default_table
# images and drawing
gauge.imread(path); gauge.imwrite(path, arr); gauge.overlay(img, result); gauge.to_rgb(img)
gauge.suggest(img)
# licence
gauge.license_status(); gauge.license_warnings(); gauge.TrialExpired
```